

Career Events Platform Project Plan

Working name: **EventMatch** (rename later e.g. Opportune, CareerLoop, NextStep, OppHub) Owner: Rohan Akode · Goal: a resume-defining, hireable full-stack + GenAI project

1. One-line pitch

The AI-powered career-event hub that aggregates every career event (hackathons, networking, startup & communication events) from across the internet into one feed, matches them to *your* resume, and lets anyone post their own event.

Career-focused only no entertainment/singing/dancing events.

2. Why this works (validated)

- **Real demand** students, grads & professionals hunt events across 6+ scattered sites (Unstop, Devfolio, MLH, Luma, Meetup, LinkedIn...).
 - **Differentiated** no single consumer app combines *aggregation + resume-matching + semantic search + posting* for the career niche.
 - **Reuses your proven skill** this is your Whofy ingestion pipeline + resume-matching engine, pointed at events.
 - **Hireable** data pipeline + AI matching + real full-stack product + top-notch UI.
-

Guiding principle: build smart, not hard

Use a **library or managed service whenever it solves a real problem** never hand-code 100-200 lines for something battle-tested code already does. Write custom code only for the parts that are actually *your* product (the aggregation logic, matching, and UX). This is how good engineers work, and it's a plus in interviews. - Auth + file storage -> **Supabase** (don't hand-roll login/JWT/reset flows or file handling; gives Google login free) - DB + vector search -> **MongoDB Atlas** (managed, no infra) - UI -> **MUI** components & templates (don't hand-code CSS) - Data fetching -> **React Query** (don't hand-manage loading/caching) - Forms -> **React Hook Form + Zod** (don't hand-validate) - Email -> **Resend** (don't run a mail server) - Scheduling -> **GitHub Actions** (don't build a cron server)

3. Feature set

Core (MVP build these first)

1. **Cross-platform aggregation** one feed of events pulled from multiple sources (APIs + scrapers), normalized & deduped.

2. **Resume -> event matching** user uploads resume -> AI recommends events that fit their skills/goals, with filters.
3. **Semantic search** search by meaning ("beginner AI hackathons in Hyderabad"), not exact keywords.
4. **Post your own event** create event -> shareable public link -> people register on the app -> organizer dashboard.

Stretch (add after MVP works)

5. **Deadline notifications** alerts + reminders for registration deadlines.
6. **Save to calendar** bookmark events + Google Calendar sync.
7. **Hackathon teammate finder** * match users by skills to form hackathon teams (the unique, social feature).
8. **Reviews / feedback** users rate events & give feedback on the app.

Two event types (both in one feed)

Type	Source	On click
Aggregated	Scraped/API from other sites	Links out to original site
Native (user-posted)	Posted on your app	Registers on your app + shareable link

4. The one AI engine (embeddings) powers 3 things

Everything runs on vector embeddings (MongoDB Atlas Vector Search you know this): - **Semantic search** query vector <-> event vectors - **Resume matching** resume vector <-> event vectors - **Deduplication** near-identical event vectors across sources = same event, merge them

One technique, three features. Strong, coherent interview story.

Note on search: No dumb keyword search. Use **hybrid search** = semantic search (meaning-based ranking via embeddings) + structured filters (exact conditions: type, city, date, online/offline). The two do different jobs and work together.

5. Tech stack (your existing skills + recruiter-friendly)

- **Frontend:** React (Vite) + **Customized Material UI (MUI)** for a top-notch UI [done] (decided for now; may revisit UI/UX later if not satisfied)
- No Tailwind (not needed) and no plain-CSS-from-scratch MUI gives a beautiful, professional UI fast and is recruiter-recognized. Great for the data-heavy feed + dashboards.
- **Trendy look comes from customizing MUI:** custom theme (palette, radius, shadows) + modern font (Inter/Poppins/Manrope) + generous whitespace + micro-animations (**Framer Motion**) + dark mode. Default MUI looks generic; customized MUI looks premium.
- Use MUI's **pre-built templates/blocks** (dashboard, sign-in, cards) to move fast don't hand-build layouts.

- **One styling system only.** Don't mix in Tailwind-based kits (e.g. 21st.dev/shadcn) clashing systems = messy code. 21st.dev is fine for *inspiration* only.
- Optional recruiter boost: **TypeScript** (only if time allows; plain JS is fine).
- **Backend:** FastAPI (Python) best fit for the AI/pipeline work
- **Database:** MongoDB Atlas + **Atlas Vector Search** events, registrations, teams, reviews + embeddings (document model fits flexible multi-source data)
- **Auth + Storage: Supabase** Auth (email/password + **Google login**) and file Storage (resume PDFs, event images). Build-smart: don't hand-roll auth or file handling.
- **AI APIs: Jina AI** (embeddings) + **Groq** (text generation) both free, high limits (see AI approach below)
- **Aggregation:** Python scrapers + official APIs, scheduled via **GitHub Actions** (like Whofy)
- **Deploy:** Vercel (frontend) + **Hugging Face Spaces** (backend) + Atlas (DB) + Supabase all free (see Deployment below)

Hybrid data setup (which service holds what)

Service	Holds	Why
Supabase	Auth (incl. Google login), file storage (resume PDFs, images)	Auth + file storage are painful in Mongo; Supabase does them near-free
MongoDB Atlas	Events, registrations, teams, reviews + embeddings/vector search	Flexible documents + you know Atlas Vector Search (Whofy)
> Glue: FastAPI verifies the Supabase auth token; store the Supabase <code>user_id</code> in the MongoDB user profile to link them. One standard integration point.		

Capacity free tiers are plenty (no scale worries)

Service	Free tier	Enough for you?
MongoDB Atlas (M0)	512 MB	Yes tens of thousands of events (even with embeddings)
Supabase	500 MB DB · 1 GB files · 50k users	Yes thousands of PDFs/images, far more users than a demo needs
Gemini	Free daily quota + caching	Yes plenty for building & demos
> You'd only upgrade if this became a real product with hundreds of thousands of events / tens of thousands of active users. Irrelevant for a portfolio project.		

AI & embeddings approach (which API, and how)

Job	Tool	Notes
Embeddings (semantic search, resume-matching, dedup)	Jina AI API	Most generous free tier; use the SAME model everywhere (bulk + runtime) so vectors are comparable
Text generation (resume parsing, event tagging)	Groq API	Fast + generous free tier. (Groq does NOT do embeddings that's why Jina handles embeddings.)
Resume PDF -> text	pdfplumber	Extract text first, then send to Groq to structure

How embedding runs (avoids limits AND server crashes): - **Bulk event embeddings** -> done in the **GitHub Actions** pipeline (7GB RAM free), stored in MongoDB. Heavy work never touches the web server. - **Runtime embeddings** (user search / resume upload) -> low volume -> **Jina API** (free tier easily covers it). - **Cache everything** embed each event once; never re-call. Keeps usage tiny and fast.

Deployment & hosting (free + reliable, no paying)

Part	Host	Why
Frontend	Vercel	Free, fast, never sleeps
Backend (FastAPI)	Hugging Face Spaces	Free, 16GB RAM (won't crash), doesn't spin down every 15 min, great for Python/AI
Database	MongoDB Atlas (free M0)	Managed, reliable
Auth + storage	Supabase (free)	Managed, reliable
Bulk embeds / aggregation	GitHub Actions (free)	Heavy work off the live server

Auto-deploy: Code lives on **GitHub**. A small **GitHub Action** (`.github/workflows/deploy.yml` + HF token secret) auto-pushes to the Hugging Face Space on every commit -> HF auto-rebuilds & redeploys. One-time ~10-min setup, then it's automatic like Vercel/Render.

Reliability practices (so it doesn't crash for real users): - Keep the web server **light** embeddings via API, not a heavy local model - **Async/background** processing for embedding newly posted events (the "Post" click returns instantly) - **Cache** AI results; **validate all input** (Pydantic); **rate-limit** posting to prevent abuse - Managed DB/auth/storage (Atlas + Supabase) = nothing fragile to maintain - HF Spaces (16GB RAM, no 15-min sleep) avoids the free-tier crash + cold-start problems **no keep-alive ping needed** (so no Render hours burned)

Cost: **Rs 0**. Only upgrade to a paid tier if it ever gets serious sustained traffic not needed for a portfolio.

Libraries to use (don't hand-code solved problems)

Problem	Library
UI components	MUI / Chakra UI
Data fetching + caching	TanStack Query (React Query)
Forms + validation	React Hook Form + Zod
Routing	React Router
Charts / dashboards	Recharts
Dates	Day.js / date-fns
MongoDB async (FastAPI)	Beanie / Motor
Backend validation	Pydantic (built into FastAPI)
Scraping	BeautifulSoup / httpx / Playwright
Resume PDF parsing	pdfplumber
Emails (deadline alerts)	Resend / SendGrid
Calendar export	.ics library / Google Calendar API

Keeping token cost near zero

- Use **Gemini Flash** (free tier) for parsing/tagging
- **Cache** embeddings & AI outputs (never re-embed the same event)
- Embed events once at ingestion, not per search
- Rate-limit AI features for demo users

6. Architecture (data flow)



```

| queries
[FastAPI backend] -- resume parse -> embed -> match
^ semantic search
|
[React frontend] feed · search · resume upload · post event · dashboards

```

7. Data sources for aggregation (start small)

- **APIs first (reliable):** Eventbrite, Meetup, Luma, Devfolio (where available)
- **Scrapers (where no API):** MLH, Unstop listings, Reskilll, college/community pages
- **Native:** user-posted events

Start with 2-3 sources for MVP, expand later. Reliability > quantity.

8. Data model (core collections)

- **events** title, description, type, date, deadline, location (city/online), source, sourceUrl, organizerId (if native), embedding[], tags[]
- **users** name, email, passwordHash, role, skills[], resumeText, resumeEmbedding[], savedEvents[]
- **registrations** userId, eventId, timestamp (for native events)
- **teams** eventId, members[], requiredSkills[], openSlots (teammate finder)
- **reviews** userId, eventId (or "app"), rating, comment

9. Build roadmap (realistic, part-time)

Phase 1 Foundation (Week 12) - Project setup (React + FastAPI + MongoDB), auth, basic layout & design system - Event data model + manual "post event" CRUD + event detail page

Phase 2 The feed & search (Week 34) - Build 1 aggregation source end-to-end (API), normalize into DB - Event feed UI with **structured filters** (type, city, date, online/offline) exact-match, NOT keyword text search - Add embeddings -> **semantic search** (meaning-based) - Combine into **hybrid search** = semantic ranking + structured filters (no dumb keyword search)

Phase 3 The AI star (Week 56) - Resume upload + parse (Gemini) -> embed -> **resume->event matching** page - Add 1-2 more aggregation sources + **semantic dedup** - Schedule refresh via GitHub Actions

Phase 4 Posting loop + polish (Week 78) - Native event posting -> shareable link -> **registration flow** -> organizer dashboard - **Save to calendar** + bookmarks - Polish the UI to top-notch (the priority!)

Phase 5 Stretch (Week 9+) - **Deadline notifications** - **Hackathon teammate finder** * - **Reviews/feedback** - Deploy, write README + architecture diagram + demo GIF

Ship a working MVP (Phases 14) before touching stretch features.

10. Resume talking points (write these)

- "Built a career-events platform that **aggregates and de-duplicates events from multiple sources** via an automated pipeline (GitHub Actions)."
- "Implemented **semantic search and resume-to-event matching** using Gemini embeddings + MongoDB Atlas Vector Search."
- "Designed a **native event-posting flow** with shareable links, registration, and an organizer dashboard."
- "Built a **skill-based hackathon teammate matcher** using vector similarity."

11. Risks & mitigations (important recruiters ask about these)

Risk	Mitigation
Scraping is fragile (sites change layout / block bots)	Prefer official APIs first; scrape only where needed; isolate each source so one breaking doesn't kill the feed
Legal / ToS of scraping	Use APIs where terms allow; respect robots.txt; attribute + link back to source; don't republish full copyrighted content
Cold start (empty app)	Aggregation fills the feed day one; seed with a few sources before launch
Duplicate events across sources	Semantic dedup with embeddings; merge near-identical events
Spam / fake user-posted events	Moderation: require login to post, basic checks, an admin review/report flag, auto-expire past events
Stale events	Auto-expire events past their date; scheduled cleanup job
AI token cost / limits	Jina + Groq free tiers; bulk embeds run in GitHub Actions; cache everything; embed once at ingestion

12. Content moderation (for user-posted events)

- Require **login to post** (no anonymous posting)
- **Report/flag** button on native events + simple admin review queue
- Basic validation (required fields, valid dates/links)
- **Auto-expire** past events so the feed stays clean

13. Definition of done (MVP)

- ☐ Aggregated feed from ≥ 2 sources, deduped
- ☐ Semantic search working
- ☐ Resume upload -> matched events
- ☐ Post event -> share link -> register
- ☐ Clean, responsive, top-notch UI
- ☐ Deployed + README with architecture diagram + demo GIF